

InboxPilot Configuration Handbook

This guide configures InboxPilot from a fresh machine through a working Render deployment. It covers MongoDB Atlas, Google Cloud Gmail OAuth, the FastAPI ML service, optional AI drafting, Render, the React frontend, local environment variables, and verification. The steps match the variable names in this repository.

Recommended order: create the MongoDB database, create the Google OAuth application, push the repository to GitHub, deploy the ML service, deploy the API, deploy the frontend, then test Gmail with a dedicated mailbox. Do not test autonomous mailbox changes against an important personal inbox.

1. What you will create

Component	Where it runs	Purpose	Required for demo mode?
React web app	Render Static Site or Vercel	Inbox UI	Yes
Express API	Render Web Service	OAuth, Gmail orchestration, policy, persistence	Yes
FastAPI ML service	Render Web Service	Classification, drafting, correction intake	No; API has a local fallback
MongoDB Atlas	MongoDB cloud	Users, decisions, settings, corrections	No; demo uses memory
Google Cloud project	Google Cloud Console	Gmail API and OAuth credentials	No; required for Gmail
Optional LLM provider	Provider dashboard	Future bounded RAG drafting	No; current draft fallback is deterministic

The Google OAuth web-server flow uses a confidential backend that can securely store refresh tokens and maintain state. Google recommends using an OAuth client library for this flow and registering exact redirect URIs.¹ The project uses the official `googleapis` Node.js library.

2. Before you begin

Install Git, Node.js 22 or newer, Python 3.11 or newer, and a GitHub account. Download the project ZIP and extract it, or clone the repository after uploading it to GitHub.

From the repository root, create a local file named `.env`. Do not name it `.env.example`, and do not commit it. The repository `.gitignore` already excludes `.env`.

Start with this template:

```
NODE_ENV=development
API_PORT=4000
WEB_ORIGIN=http://localhost:5173
MONGODB_URI=
ML_SERVICE_URL=http://localhost:8000
SESSION_SECRET=replace-with-a-long-random-secret
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GOOGLE_REDIRECT_URI=http://localhost:4000/api/gmail/callback
GOOGLE_ALLOWED_EMAIL=your-email@gmail.com
OPENAI_API_KEY=
```

Generate a strong session secret. On macOS or Linux, run `openssl rand -base64 32`. On Windows PowerShell, run `[Convert]::ToBase64String((1..32 | ForEach-Object { Get-Random -Maximum 256 })))`. Paste the output into `SESSION_SECRET`.

Install dependencies and run the baseline checks:

```
npm install
npm test -w apps/api
npm run build -w apps/web
cd services/ml
python -m pip install -r requirements.txt
pytest
```

3. MongoDB Atlas configuration

MongoDB Atlas separates your Atlas login from your database user. The Atlas login manages the cloud project. The database user is what the Express API uses to connect to the cluster.[2](#)

3.1 Create an Atlas project

1. Open [MongoDB Atlas](#).
2. Sign in or create an account.
3. If Atlas asks for an organization, create one named `InboxPilot`.
4. In the left navigation, open **Projects**.
5. Click **New Project**.
6. Enter `InboxPilot` as the project name.
7. Click **Next** or **Create Project**.

If you land on the project overview instead of the database screen, use the left navigation and choose **Database** or **Clusters**.

3.2 Create a database deployment

1. On the **Database** or **Clusters** page, click **Build a Database**.
2. Choose the free shared option if it is available to you. The label may appear as **Free**, **M0**, or **Shared** depending on the current Atlas UI.
3. Choose a cloud provider and a region close to Render's API region. Oregon is a practical default if you use Render's default region.
4. Set the cluster name to `inboxpilot-cluster`.
5. Click **Create** or **Deploy**.

Atlas may ask for a database username and password during this process. If so, create a dedicated database user named `inboxpilot-app`, use **Autogenerate Secure Password**, and copy the password into your password manager immediately. Do not use your Atlas account password.

If Atlas does not ask for a database user, continue to the next section. The **Connect** dialog can create one later.²

3.3 Add a database user

1. In the left navigation, open **Security** → **Database Access**.
2. Click **Add New Database User**.
3. Choose **Password** authentication.
4. Enter username `inboxpilot-app`.

5. Generate a long password.
6. Under database privileges, choose the smallest useful role for this app. **Read and write to any database** is acceptable for an individual portfolio deployment. Do not choose the Atlas administration role for the application.
7. Click **Add User**.

The database username and password are different from your Atlas website login.[2](#)

3.4 Configure network access

1. In the left navigation, open **Security** → **Network Access**.
2. Click **Add IP Address**.
3. For a temporary local development test, click **Add My Current IP Address**.
4. For Render, Render's outbound IPs may not be fixed on all plans. If Atlas requires a broad allow-list for your setup, use `0.0.0.0/0` only with a strong database password and least-privilege database user. This permits connections from anywhere and is less secure.
5. Click **Confirm**.

For a more restrictive production setup, use a Render plan and networking arrangement that provides stable outbound IPs or private networking. Atlas requires your application network to be present in the project IP access list.[2](#)

3.5 Copy the connection string

1. Return to **Database** → **Clusters**.
2. Find `inboxpilot-cluster`.
3. Click **Connect**.
4. Choose **Drivers** or **Connect your application**.
5. Choose **Node.js** and the current driver version.
6. Copy the connection string. It should resemble:

```
mongodb+srv://inboxpilot-app:<PASSWORD>@inboxpilot-cluster.xxxxx.mongodb.net/
```

Replace `<PASSWORD>` with the database-user password. Add the database name `inboxpilot` after `.net/`:

```
mongodb+srv://inboxpilot-app:URL_ENCODED_PASSWORD@inboxpilot-cluster.xxxx:
```

If the password contains `@`, `/`, `:`, `?`, `#`, or `%`, URL-encode it. The simplest route is to generate a password containing only letters, numbers, and a few safe symbols. Paste the final value into local `.env`:

```
MONGODB_URI=mongodb+srv://inboxpilot-app:YOUR_PASSWORD@inboxpilot-cluster.xxx
```

The API creates the `users` and `decisions` collections when the application first writes data. You can inspect them under **Database** → **Browse Collections** after the first successful production sync.

4. Google Cloud and Gmail OAuth configuration

InboxPilot needs the Gmail API and a Web application OAuth client. The application requests `https://www.googleapis.com/auth/gmail.modify` because it reads messages and can modify labels or create drafts. This is a sensitive Gmail scope, so keep the app in testing while developing and add only your own test account. Google recommends minimum necessary scopes and notes that sensitive or restricted scopes can require additional review for public release.³

4.1 Create or select a Google Cloud project

1. Open [Google Cloud Console](#).
2. At the top of the page, click the project selector. It may display **Select a project** or an existing project name.
3. Click **New Project**.
4. Set the project name to `InboxPilot`.
5. Click **Create**.
6. Reopen the project selector and select the new `InboxPilot` project.

If the left navigation is collapsed, click the three-line menu icon in the upper-left corner.

4.2 Enable the Gmail API

1. Open **APIs & Services** → **Library**.
2. In the search box, type `Gmail API`.
3. Select **Gmail API** from Google.
4. Click **Enable**.
5. Wait until the API overview appears.

The API Library is also reachable directly at [Google API Library](#). Google's OAuth setup requires APIs to be enabled before credentials are used.¹

4.3 Configure the OAuth consent screen

Google's current UI may call this **Google Auth Platform** instead of **OAuth consent screen**. Open **Google Auth Platform** → **Branding**. If you cannot find it, use the top search bar and search for `Google Auth Platform` or `OAuth consent screen`.

If you see **Google Auth platform not configured yet**:

1. Click **Get Started**.
2. Under **App Information**, enter app name `InboxPilot`.
3. Choose a **User support email** that you can access.
4. Click **Next**.
5. For a personal Gmail account, choose **External**.
6. Click **Next**.
7. Enter your developer/contact email.
8. Accept the Google API Services User Data Policy.
9. Click **Create** or **Continue**.

Google's current flow separates the configuration into **Branding**, **Audience**, and **Data Access**.³

4.4 Add yourself as a test user

1. Open **Google Auth Platform** → **Audience**.
2. Find **Test users**.
3. Click **Add users**.
4. Add the Gmail account you will use for testing.

5. Click **Save**.

If you forget this step for an External testing app, Google may show an access-blocked or app-not-verified message when you try to connect.

4.5 Configure Gmail data access

1. Open **Google Auth Platform** → **Data Access**.
2. Click **Add or Remove Scopes**.
3. Search for Gmail or paste this scope:

```
https://www.googleapis.com/auth/gmail.modify
```

1. Select the scope.
2. Click **Update** or **Save**.

The application should not request Gmail send permission. InboxPilot saves drafts but does not send messages automatically.

4.6 Create the OAuth client ID

1. Open **Google Auth Platform** → **Clients**, or open **APIs & Services** → **Credentials**.
2. Click **Create Client** or **Create Credentials** → **OAuth client ID**.
3. Choose **Web application** as the application type.
4. Enter the name `InboxPilot Web Backend`.
5. Under **Authorized JavaScript origins**, add your frontend origin if Google asks for it:

```
http://localhost:5173
```

For production, add the exact frontend URL, such as:

```
https://inboxpilot-web.onrender.com
```

1. Under **Authorized redirect URIs**, add the local callback:

```
http://localhost:4000/api/gmail/callback
```

1. Later, add the production callback:

```
https://YOUR_API_SERVICE.onrender.com/api/gmail/callback
```

Replace `YOUR_API_SERVICE` with the actual Render API service name. Do not add a trailing slash unless the application URL also contains one. Google redirect URIs must match exactly.[1](#)

1. Click **Create**.
2. Copy the **Client ID** and **Client Secret**.

Do not commit the downloaded client secret JSON file. This project uses the two environment variables instead:

```
GOOGLE_CLIENT_ID=your-client-id.apps.googleusercontent.com  
GOOGLE_CLIENT_SECRET=your-client-secret  
GOOGLE_REDIRECT_URI=http://localhost:4000/api/gmail/callback  
GOOGLE_ALLOWED_EMAIL=your-test-gmail-address@gmail.com
```

4.7 Test local OAuth

Start the API and web app:

```
npm run dev
```

Open the frontend at `http://localhost:5173`. The current UI is demo-first, so if you do not see a visible Gmail connect button, open this URL directly in the browser:

```
http://localhost:4000/api/gmail/connect
```


Expected behavior:

1. The browser redirects to Google.
2. Google shows the InboxPilot consent screen.
3. You select the test account.
4. You review the Gmail permission.
5. You click **Continue** or **Allow**.
6. Google redirects to `/api/gmail/callback`.
7. The API stores the refresh token in MongoDB and redirects to the frontend.

If Google displays **Access blocked: InboxPilot has not completed the Google verification process**, confirm that the Gmail address appears under **Google Auth Platform** → **Audience** → **Test users**.

If Google displays **redirect_uri_mismatch**, copy the callback URL from the browser error and add that exact URL under the OAuth client's **Authorized redirect URIs**.

5. ML and AI configuration

5.1 Current ML service

The project includes a Python FastAPI service at `services/ml`. The current champion is trained from 125 curated examples across five classes using word and character TF-IDF, transparent intent markers, balanced logistic regression, and calibrated probabilities. Its current five-fold cross-validation metrics are 74.4% accuracy, 73.8% macro-F1, and 74.4% balanced accuracy. These are still starter-corpus metrics, not a guarantee for your mailbox. It has two model modes:

Mode	Trigger	Behavior
Champion artifact	<code>services/ml/models/champion.joblib</code> exists	Uses the trained TF-IDF/logistic-regression model
Safe fallback	Artifact unavailable	Uses conservative keyword/rule classification

Train the local champion artifact:

```
cd services/ml
python train.py
pytest
```

The training command creates:

```
services/ml/models/champion.joblib
services/ml/models/metrics.json
```

Start the service locally:

```
uvicorn app.main:app --reload --port 8000
```

Open `http://localhost:8000/health`. You should see `trainedModelAvailable: true` after training.

Test classification:

```
curl -X POST http://localhost:8000/classify \
  -H "Content-Type: application/json" \
  -d '{"subject":"Project update","snippet":"Can we review tomorrow?","se
```

5.2 Render ML service

The repository's `render.yaml` deploys the ML service from `services/ml`.

Use these values if creating it manually:

Render field	Value
Service type	Web Service
Runtime	Python
Root Directory	services/ml
Build Command	pip install -r requirements.txt && python train.py
Start Command	uvicorn app.main:app --host 0.0.0.0 --port \$PORT
Health Check Path	/health

After deployment, open:

```
https://YOUR_ML_SERVICE.onrender.com/health
```

Copy the ML URL into the API service:

```
ML_SERVICE_URL=https://YOUR_ML_SERVICE.onrender.com
```

The Express API calls `/classify` with a four-second timeout. If the ML service sleeps, fails, or returns a non-2xx response, the API uses the safe local fallback and logs the fallback event.

5.3 Optional external AI provider

The current production-safe draft path is deterministic and does not require an AI key. The `OPENAI_API_KEY` variable is reserved for a later bounded RAG drafting integration. Do not add a provider key until you want to enable that feature.

If you later configure an LLM provider, use the provider's server-side secret storage. Put the key only in the Render API service environment, never in React and never in `VITE_*` variables. A key beginning with `VITE_` is exposed to the browser bundle.

The desired production behavior is:

1. Retrieve a few redacted sent-email examples.
2. Send only bounded context to the model.
3. Mark email content as untrusted data.

4. Generate a draft, never a sent message.
5. Show grounding references to the user.
6. Require explicit approval before saving a Gmail draft.

6. Render deployment through the UI

Render Blueprints use the `render.yaml` file at the repository root. Render supports `buildCommand`, `startCommand`, `healthCheckPath`, `rootDir`, and environment variables in a Blueprint.[4](#)

6.1 Push to GitHub

1. Create a new GitHub repository named `inboxpilot`.
2. Extract the project bundle.
3. Open a terminal in the extracted folder.
4. Run:

```
git init
git add .
git commit -m "Build InboxPilot autonomous email triage agent"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/inboxpilot.git
git push -u origin main
```

Before pushing, check that `.env` is not listed in `git status`. Never upload Gmail credentials, MongoDB passwords, or an OAuth client secret.

6.2 Create the Render Blueprint

1. Open [Render Dashboard](#).
2. Click **New** in the upper-right corner.
3. Choose **Blueprint**.
4. Connect GitHub if Render asks for permission.
5. Select the `inboxpilot` repository.
6. Select the `main` branch.

7. Render should detect `render.yaml` at the repository root.
8. Review the services before clicking **Apply**.
9. If Render asks for values marked `sync: false`, leave the page open; you will enter them in the service Environment pages.

The Blueprint creates an API service, an ML service, and a static web service.

6.3 Configure API environment variables in Render

1. Open the service whose name ends in `-api`.
2. In the left sidebar, click **Environment**.
3. Under **Environment Variables**, click **Add Environment Variable**. Render's current UI also provides **Add from .env** for bulk input.[5](#)
4. Add these keys:

```
NODE_ENV=production
WEB_ORIGIN=https://YOUR_WEB_SERVICE.onrender.com
MONGODB_URI=mongodb+srv:// ...
ML_SERVICE_URL=https://YOUR_ML_SERVICE.onrender.com
SESSION_SECRET=use-a-new-long-random-secret
GOOGLE_CLIENT_ID= ...
GOOGLE_CLIENT_SECRET= ...
GOOGLE_REDIRECT_URI=https://YOUR_API_SERVICE.onrender.com/api/gmail/callback
GOOGLE_ALLOWED_EMAIL=your-test-email@gmail.com
```

1. Click **Save, rebuild, and deploy**.

Render's **Save only** option does not update a running service until a later deploy. Use **Save, rebuild, and deploy** when changing build-time or runtime configuration.[5](#)

6.4 Configure ML environment variables

The ML service needs no secret variables for the current version. Verify:

```
Build: pip install -r requirements.txt && python train.py
Start: uvicorn app.main:app --host 0.0.0.0 --port $PORT
Health: /health
```

6.5 Configure the frontend environment variable

1. Open the service whose name ends in `-web`.
2. Click **Environment**.
3. Add:

```
VITE_API_URL=https://YOUR_API_SERVICE.onrender.com/api
```

1. Click **Save, rebuild, and deploy**.

Because this value is compiled into the browser bundle, the frontend must be rebuilt after changing it.

6.6 Update Google OAuth with Render URLs

After Render creates the services, copy their public URLs. Return to Google Cloud:

1. Open **Google Auth Platform** → **Clients**.
2. Open the InboxPilot Web Backend client.
3. Add the production frontend origin under **Authorized JavaScript origins**.
4. Add the production API callback under **Authorized redirect URIs**.
5. Click **Save**.

Use the exact URLs. Do not use the frontend URL as the callback. The callback belongs to the API service.

6.7 Verify Render deployment

Open these URLs in a browser:

```
https://YOUR_ML_SERVICE.onrender.com/health
https://YOUR_API_SERVICE.onrender.com/api/health
https://YOUR_WEB_SERVICE.onrender.com
```

The API health response should include:

```
{
  "status": "ok",
  "service": "api",
  "dbConnected": true,
  "mlConfigured": true
}
```

If `dbConnected` is false, inspect the API logs and verify the Atlas URI and IP access list. If `mlConfigured` is false, verify the API service's `ML_SERVICE_URL` variable.

7. Full production smoke test

Use a dedicated Gmail account or a mailbox with disposable test messages.

1. Open the production frontend.
2. Start Gmail connection through the API connect route if the UI does not yet display a connect control:

```
https://YOUR_API_SERVICE.onrender.com/api/gmail/connect
```

1. Complete Google consent.
2. Return to the frontend.
3. Click **Analyze inbox**.
4. Confirm that decisions appear in MongoDB Atlas under **Database** → **Browse Collections** → **decisions**.
5. Confirm a low-risk promotional test message is archived only if the threshold permits it.
6. Confirm the **Undo archive** action restores the inbox label.

7. Confirm a routine test message produces a draft suggestion.
8. Approve the draft and verify that Gmail contains a draft, not a sent message.
9. Send a security/password/payment test message and confirm it is escalated and untouched.
10. Use **Correct label** and confirm the decision document receives a **correction** field.

8. Troubleshooting by symptom

Symptom	Likely cause	Fix
<code>redirect_uri_mismatch</code>	Google callback differs by scheme, host, port, or path	Add the exact callback shown in the error to the OAuth client
Access blocked	Gmail account is not a test user	Add it under Google Auth Platform → Audience → Test users
<code>invalid_client</code>	Client ID and secret belong to different projects	Re-copy both values from the same OAuth client
API says <code>dbConnected:false</code>	Atlas URI, password, database name, or IP access is wrong	Check <code>MONGODB_URI</code> , URL-encode password, and Atlas Network Access
API says <code>mlConfigured:false</code>	API environment lacks ML URL	Add <code>ML_SERVICE_URL</code> and redeploy
ML <code>/health</code> is 404	Wrong Render root directory or start command	Root must be <code>services/ml</code> ; use the documented Uvicorn command
Browser CORS error	<code>WEB_ORIGIN</code> does not exactly match frontend origin	Set the API variable to the full frontend origin with no <code>/api</code> suffix
Frontend uses old API URL	Vite variable changed without rebuild	Save with rebuild and deploy
Render service sleeps	Free instance cold start	Wait for the first request, use fallback behavior, or upgrade the service
Gmail callback succeeds but later sync is disconnected	Cookies blocked or missing HTTPS frontend/API setup	Use the Render HTTPS URLs and confirm browser requests include credentials

Symptom	Likely cause	Fix
Mongo password contains special characters	URI parsing breaks	URL-encode the password or create a safe generated password
App appears to do nothing	API URL is empty or unreachable	Open browser DevTools → Network and inspect the failing request URL

9. Security checklist before sharing the app

Use a dedicated Gmail test account first. Keep `GOOGLE_CLIENT_SECRET` , `MONGODB_URI` , `SESSION_SECRET` , and any AI key only in Render Environment Variables or a local ignored `.env` . Rotate any credential that was pasted into a public repository, issue, screenshot, or chat. Do not enable automatic email sending. Keep Gmail scope at `gmail.modify` until the product is stable. Do not allow unrestricted unsubscribe requests without a separate review flow. Keep MongoDB users least-privileged and remove temporary `0.0.0.0/0` access if your deployment supports a narrower network configuration.

10. References